

CHAPTER 1

ABSTRACT

SafeByte is a secure data encryption tool designed to provide users with a robust and user-friendly platform for encrypting and decrypting sensitive information. Utilizing the Advanced Encryption Standard (AES) in Cipher Block Chaining (CBC) mode, SafeByte ensures the confidentiality and integrity of both plaintext and files through strong cryptographic methods.

The tool offers a range of features, including file encryption, text encryption with output in Hex and Base64 formats, and a real-time chat application that enables secure communication over networks.

Additionally, SafeByte incorporates data compression to optimize storage and transmission efficiency. With a focus on usability, the tool includes comprehensive input validation and clear user prompts, making it accessible for both technical and non-technical users. SafeByte represents a reliable solution for individuals and organizations seeking to protect their sensitive data against unauthorized access and ensure secure communication channels.

CHAPTER 2

INTRODUCTION

2.1 DESCRIPTION:

SafeByte is a cutting-edge data encryption tool engineered to provide robust and user-friendly solutions for safeguarding sensitive information. By leveraging the Advanced Encryption Standard (AES) in Cipher Block Chaining (CBC) mode, SafeByte ensures the confidentiality and integrity of both plaintext data and files, employing advanced cryptographic techniques for enhanced security.

2.2 EXISTING SYSTEM:

- i. **Complexity for Non-Technical Users:** Many encryption tools have complicated interfaces, making them difficult for non-technical users to navigate and utilize effectively.
- ii. **Limited Features:** Existing systems often focus on either file encryption or text encryption, lacking an integrated platform that supports functionalities like secure communication.
- iii. **Vulnerability to Attacks:** Some systems do not implement strong encryption standards or fail to provide adequate input validation, leaving data susceptible to unauthorized access or manipulation.

2.3 PROBLEM DEFINITION:

In today's digital landscape, the growing reliance on electronic communication and data storage has increased the risk of sensitive information being intercepted, accessed, or compromised by unauthorized entities. While

encryption is a proven method to ensure data security, many existing solutions suffer from several limitations that hinder their effectiveness and usability:

- i. **Complexity and Accessibility:** Many encryption tools are designed with a steep learning curve, making them inaccessible to non-technical users. This limits their adoption in personal and small-business scenarios where simplicity and usability are key.
- ii. **Fragmented Solutions:** Current tools often focus on a single functionality, such as text or file encryption, and lack an all-in-one platform that integrates encryption, secure communication.

2.4 PROPOSED SYSTEM:

SafeByte addresses the limitations of existing encryption tools by providing a secure, user-friendly solution designed for data protection.

- i. **Advanced Encryption:** Implements the Advanced Encryption Standard (AES) in Cipher Block Chaining (CBC) mode to ensure strong security and data integrity for both files and plaintext.
- ii. **User-Friendly Interface:** Features an intuitive design with clear prompts and comprehensive input validation, making it accessible to both technical and non-technical users.
- iii. **Integrated Functionality:** Combines multiple features into one platform, including:
 - a. File encryption and decryption.
 - b. Text encryption with Hexadecimal and Base64 output formats.
 - c. Secure real-time communication through an encrypted chat application.

CHAPTER 3

GENERAL BLOCK DIAGRAM

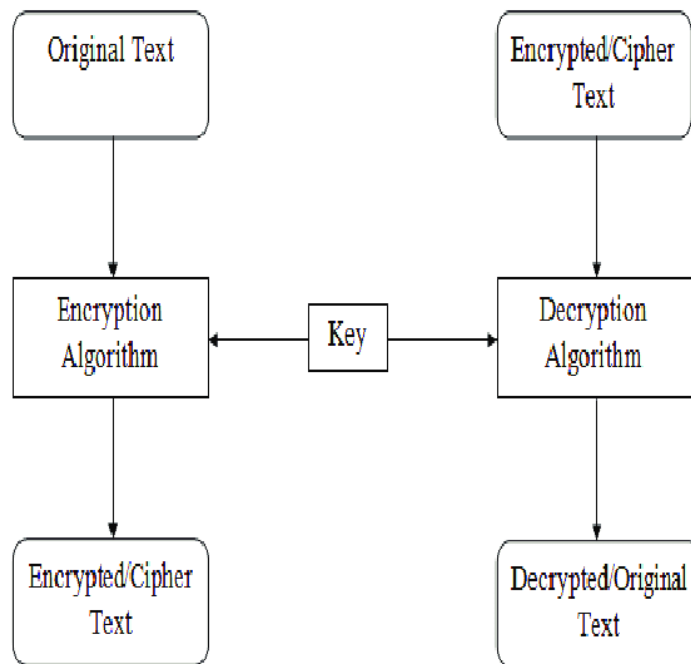


FIG 3.0 BLOCK DIAGRAM

3.1 ORIGINAL TEXT:

HELLO WORLD

FIG 3.1 ORIGINAL TEXT

Description: This represents the input data or plaintext that needs to be secured. It could be any sensitive information such as passwords, personal data, or messages.

Purpose: To serve as the starting point for encryption. This data is in human-readable format and is vulnerable to unauthorized access.

3.2 ENCRYPTION ALGORITHM:



FIG 3.2 AES ENCRYPTION ALGORITHM

Description: This block represents the mathematical process used to transform the plaintext into ciphertext. In the case of AES, this involves steps like SubBytes, ShiftRows, MixColumns, and AddRoundKey operations.

Key Input: The algorithm requires a secret key as input to ensure secure encryption.

Output: The result of this block is ciphertext, which appears as scrambled, unreadable data.

3.3 KEY:

- Round 0: 54 68 61 74 73 20 6D 79 20 4B 75 6E 67 20 46 75
- Round 1: E2 32 FC F1 91 12 91 88 B1 59 E4 E6 D6 79 A2 93
- Round 2: 56 08 20 07 C7 1A B1 8F 76 43 55 69 A0 3A F7 FA
- Round 3: D2 60 0D E7 15 7A BC 68 63 39 E9 01 C3 03 1E FB
- Round 4: A1 12 02 C9 B4 68 BE A1 D7 51 57 A0 14 52 49 5B
- Round 5: B1 29 3B 33 05 41 85 92 D2 10 D2 32 C6 42 9B 69
- Round 6: BD 3D C2 B7 B8 7C 47 15 6A 6C 95 27 AC 2E 0E 4E
- Round 7: CC 96 ED 16 74 EA AA 03 1E 86 3F 24 B2 A8 31 6A
- Round 8: 8E 51 EF 21 FA BB 45 22 E4 3D 7A 06 56 95 4B 6C
- Round 9: BF E2 BF 90 45 59 FA B2 A1 64 80 B4 F7 F1 CB D8
- Round 10: 28 FD DE F8 6D A4 24 4A CC C0 A4 FE 3B 31 6F 26

FIG 3.3 SAMPLE KEY

Description: The secret key is a critical component for both encryption and decryption. It determines the output of the encryption algorithm and must be kept secure.

Purpose: To ensure that only authorized parties with the correct key can decrypt the data.

3.4 ENCRYPTED/CIPHER TEXT:

2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c

FIG 3.4 SAMPLE ENCRYPTED/CIPHER TEXT

Description: This is the output of the encryption algorithm. It is an unintelligible version of the original text, designed to protect the information during storage or transmission.

Purpose: To prevent unauthorized access to sensitive information. Without the correct key, the ciphertext is useless to attackers.

3.5 DECRYPTION ALGORITHM:



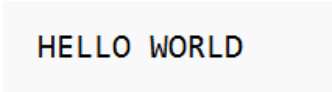
FIG 3.5 AES DECRYPTION ALGORITHM

Description: This block performs the reverse operation of the encryption algorithm. It transforms the ciphertext back into plaintext using the same key that was used for encryption.

Key Input: The decryption algorithm requires the correct secret key to successfully decrypt the ciphertext.

Purpose: To retrieve the original data for authorized users, ensuring the confidentiality and integrity of the information.

3.6 DECRYPTED/ORIGINAL TEXT:



HELLO WORLD

FIG 3.6 DECRYPTED/ORIGINAL TEXT

Description: This represents the final output of the decryption algorithm. It is identical to the original plaintext provided at the start of the process.

Purpose: To ensure that the intended recipient or system can access the original data in its readable form.

3.7 PROCESS OVERVIEW:

Encryption Flow: The original text is input into the encryption algorithm along with a key. The algorithm transforms it into ciphertext to secure the data.

Transmission/Storage: The encrypted data is transmitted or stored in its secure form.

Decryption Flow: The ciphertext is input into the decryption algorithm along with the same key. The algorithm restores it to its original form as plaintext.

CHAPTER 4

FLOWCHAT

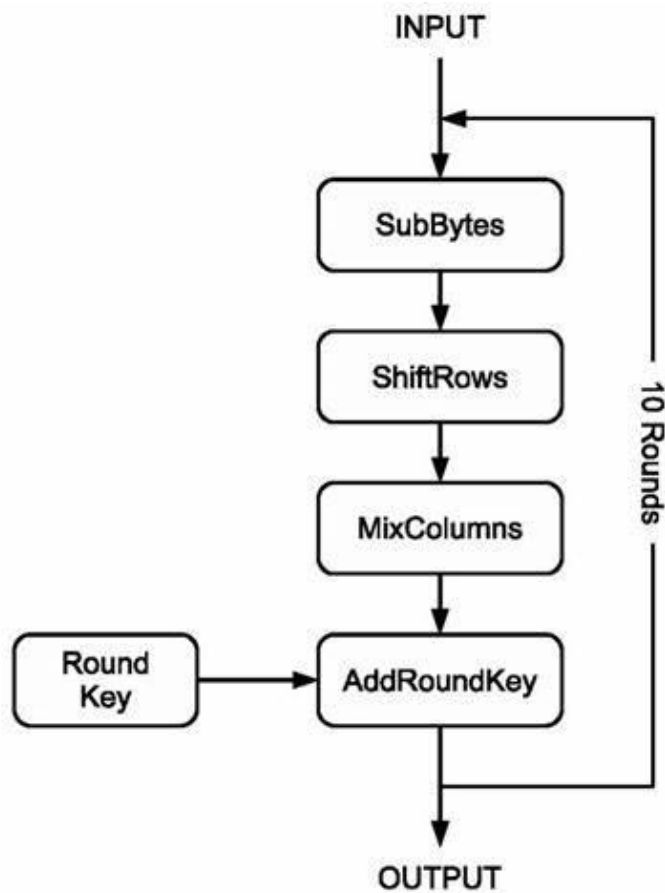


FIG 4.1 FLOWCHART

4.1 SUB BYTES:

This step provides non-linearity in the encryption process by substituting bytes using a substitution box (S-Box).

Process:

- Each byte of the state matrix is replaced with its corresponding value from a pre-defined S-Box.

- The S-Box is constructed using mathematical functions over a finite field (Galois Field $GF(2^8)$).

Key Properties:

- Ensures confusion by replacing input bytes with unrelated substitutes.
- Prevents simple relationships between plaintext and ciphertext.

4.2 SHIFTRROWS:

Introduces diffusion by permuting the positions of bytes within the state matrix.

Process:

- Rows of the state matrix are shifted cyclically to the left by a specific offset:
 - i. Row 0: No shift.
 - ii. Row 1: Shift by 1 position.
 - iii. Row 2: Shift by 2 positions.
 - iv. Row 3: Shift by 3 positions.
- This ensures that columns are mixed more thoroughly in subsequent steps.

Key Properties:

- Mixes data across rows, contributing to diffusion.

4.3 MIXCOLUMNS:

Applies a mathematical transformation to mix the bytes within each column of the state matrix, further increasing diffusion.

Process:

- Each column is treated as a polynomial and multiplied by a fixed polynomial $c(x) = 03x^3 + 01x^2 + 01x + 02$ in $GF(2^8)$.
- This step involves matrix multiplication with a fixed matrix defined in AES standards.

Key Properties:

- Ensures that a change in one byte affects all bytes in the column.

4.4 ADDROUNDKEY:

Combines the current state matrix with the round key using a bitwise XOR operation.

Process:

- A round key (derived from the main encryption key) is XORed with the current state matrix.
- This operation is the only step in AES where the encryption key directly interacts with the plaintext or intermediate state.

Key Properties:

- Ensures key dependency for every round.
- Provides security by obscuring the relationship between plaintext and ciphertext.

4.5 ROUND KEY:

A unique subkey is derived from the main encryption key for each round of AES.

Process:

- AES uses the key expansion algorithm to generate round keys.
- These keys are computed using the original encryption key, a round constant, and the S-Box.

Key Properties:

- Guarantees a new, unique key for each round, adding complexity to the encryption process.

4.6 10 ROUNDS (REPETITION):

AES performs multiple rounds of the above operations to enhance security.

Details:

- Number of Rounds:
 1. AES-128: 10 rounds.
 2. AES-192: 12 rounds.
 3. AES-256: 14 rounds.
- Each round consists of SubBytes, ShiftRows, MixColumns, and AddRoundKey.
- The final round omits the MixColumns step to complete the encryption process.

4.7 OUTPUT:

Produces the final ciphertext after all rounds are completed.

Details:

- The resulting matrix after the last round is converted into a single block of encrypted data (ciphertext).

CHAPTER 5

REQUIREMENTS AND EXPLANATION

5.1 REQUIREMENTS:

i. SOFTWARE REQUIREMENTS:

a. Operating System:

- Windows 7 or later

b. Programming Environment:

- Python Version: Python 3.8 or later

c. Required Python Libraries:

- **pycryptodome:** Provides cryptographic algorithms like AES for encryption and decryption.
- **colorama:** Used for colored text output in the terminal for better readability.
- **zlib:** Built-in library for compression and decompression.
- **socket:** Built-in library for network communication.
- **threading:** Built-in library to handle multi-threaded server communication.

d. Networking Requirements:

- TCP/IP Protocol Support for chat functionality.
- Open Port (e.g., 9999 by default) for server communication.

e. Installation of Dependencies:

- Install required libraries using pip if not already installed:

“pip install pycryptodome colorama”

f. Additional Requirements:

- **Text Editor/IDE:** VS Code, PyCharm, or any preferred text editor.
- **Admin Rights:** May be required to open ports for the chat application on the server side.

5.2 EXPLANATION:

1. DEPENDENCIES:

- **Crypto.Cipher.AES:** Provides AES encryption and decryption.
- **Crypto.Random.get_random_bytes:** Generates secure random bytes for keys and initialization vectors (IVs).
- **Crypto.Protocol.KDF.scrypt:** Derives cryptographic keys from passwords for better security.
- **base64 and binascii:** Handle encoding and decoding of binary data into Base64 and hexadecimal.
- **zlib:** Compresses and decompresses data.
- **socket and threading:** Enable networking functionality for encrypted chat.
- **colorama:** Adds coloured output for enhanced readability.

2. KEY FEATURES:

Key and IV Generation:

- **generate_key_iv:** Generates a 128-bit AES key and a 128-bit IV using secure random bytes.

- **generate_key_from_password:** Derives a cryptographic key from a password using the script key derivation function.

Data Padding and Unpadding:

AES operates on fixed-size blocks of data (16 bytes for AES). Padding ensures that data aligns with this requirement:

- **pad_data:** Pads plaintext using PKCS#7 padding.
- **unpad_data:** Removes padding after decryption.

Encryption and Decryption:

- **encrypt:** Encrypts plaintext using the specified AES mode (default: CBC) with a provided key and IV. The IV is prepended to the ciphertext.
- **decrypt:** Decrypts ciphertext by extracting the IV and decrypting the remaining ciphertext.

File Encryption/Decryption:

- **encrypt_file:** Reads and compresses the file, encrypts the data, and writes the encrypted content to a new file.
- **decrypt_file:** Reads the encrypted file, decrypts it, decompresses the data, and writes the original content back to a file.

Compression and Decompression:

- **compress_data:** Compresses data to reduce size before encryption.
- **decompress_data:** Decompresses encrypted data after decryption.

Utility Functions:

- **save_key_iv and load_key_iv:** Save and load keys and Ivs from a file in hexadecimal format.

- **show_progress:** Displays a simple progress indicator (e.g., during encryption or decryption).
- **input_with_validation:** Validates user inputs (e.g., hexadecimal strings or file paths).

3. USER INTERACTION FEATURES:

Text Encryption and Decryption:

- **encrypt_text:** Prompts the user to input plaintext, generates a key and IV, and displays the ciphertext in both Hex and Base64 formats.
- **decrypt_text:** Allows the user to decrypt a Base64-encoded ciphertext using a provided key and IV.

File Encryption and Decryption:

- Users can encrypt files, save the key and IV to a file, and later decrypt them with the saved credentials.

Chat Functionality:

The program includes a basic chat application where messages are encrypted before being sent:

- **Server (start_server):** Listens for incoming connections and decrypts received messages.
- **Client (chat_client):** Sends encrypted messages with a new IV for each message.

Help System:

- The `show_help` function explains the tool's features, commands, and usage in a user-friendly manner.

4. ENCRYPTION MODES:

The program uses the CBC (Cipher Block Chaining) mode for AES, which requires both a key and IV for encryption and decryption. This mode ensures that identical plaintext blocks produce different ciphertext, enhancing security.

5. PROGRAM STRUCTURE:

- **Main Menu (main):** Allows users to choose between encrypting text, decrypting text, encrypting files, decrypting files, using the chat application, or exiting.
- **Modularity:** Functions are modular and reusable, ensuring the code is easy to maintain and extend.

6. SECURITY CONSIDERATIONS:

- **Random Key/IV:** Ensures that every encryption operation is unique.
- **Key Derivation:** Scrypt is a strong key derivation function that adds computational cost to prevent brute-force attacks.
- **Compression Before Encryption:** Adds another layer of obfuscation and reduces file sizes.

CHAPTER 6

IMPLEMENTATION AND OUTPUT

IMPLEMENTATION:

1. **Encrypt Text:** Generate a key and IV, encrypt plaintext, and display results in Hex/Base64.
2. **Decrypt Text:** Provide ciphertext, key, and IV to retrieve the original plaintext.
3. **Encrypt Files:** Compress and encrypt files, optionally save key/IV.
4. **Decrypt Files:** Decrypt and decompress files to retrieve original content.
5. **Encrypted Chat:** Secure real-time communication with automatic encryption for messages.
6. **Help & Exit:** View usage instructions or exit the tool.

OUTPUT:

```
=====
          AES Encryption/Decryption Tool
=====
1. Encrypt Text
2. Decrypt Text
3. Encrypt/Decrypt File
4. Chat Application
5. Help
6. Exit
Enter choice (1/2/3/4/5/6): █
```

FIG 6.1 MAIN INTERFACE

```

=====
                AES Encryption/Decryption Tool
=====
1. Encrypt Text
2. Decrypt Text
3. Encrypt/Decrypt File
4. Chat Application
5. Help
6. Exit
Enter choice (1/2/3/4/5/6): 1

=====
                Encryption
=====
Enter plaintext to encrypt: HELLO
Encrypting...

Encryption Details:
Encryption Key (Hex): 10ec74b87cf1db7e349aa4eda92ffe4f
Encryption IV (Hex): a52c3517c5f52f93faeb8b942bbeceb4
Ciphertext (Hex): a52c3517c5f52f93faeb8b942bbeceb48bcb2345fad7ea6fedbd4d17744cc177
Ciphertext (Base64): pSw1F8X1L5P664uUK770tIvLI0X61+pv7b1NF3RMwXc=

```

FIG 6.2 TEXT ENCRYPTION

```

=====
                AES Encryption/Decryption Tool
=====
1. Encrypt Text
2. Decrypt Text
3. Encrypt/Decrypt File
4. Chat Application
5. Help
6. Exit
Enter choice (1/2/3/4/5/6): 2

=====
                Decryption
=====
Enter ciphertext (Base64) to decrypt: pSw1F8X1L5P664uUK770tIvLI0X61+pv7b1NF3RMwXc=
Enter encryption key (Hex): 10ec74b87cf1db7e349aa4eda92ffe4f
Enter encryption IV (Hex): a52c3517c5f52f93faeb8b942bbeceb4
Decrypting...

Decryption Result:
Decrypted Plaintext: HELLO

```

FIG 6.3 TEXT DECRYPTION

```

=====
          AES Encryption/Decryption Tool
=====
1. Encrypt Text
2. Decrypt Text
3. Encrypt/Decrypt File
4. Chat Application
5. Help
6. Exit
Enter choice (1/2/3/4/5/6): 3

=====
          File Encryption/Decryption
=====
Encrypt or decrypt file? (e/d): e
Enter file path: \Downloads\Programming\Python\AES Tool\Sample_Text.txt
Enter path to save key and IV (or press Enter to skip): \Downloads\Programming\Python\AES Tool\Save_iv_key.txt
Encrypting file...
File encrypted and saved as \Downloads\Programming\Python\AES Tool\Sample_Text.txt.enc

```

FIG 6.4 FILE ENCRYPTION

```

=====
          AES Encryption/Decryption Tool
=====
1. Encrypt Text
2. Decrypt Text
3. Encrypt/Decrypt File
4. Chat Application
5. Help
6. Exit
Enter choice (1/2/3/4/5/6): 3

=====
          File Encryption/Decryption
=====
Encrypt or decrypt file? (e/d): d
Enter file path: \Downloads\Programming\Python\AES Tool\Sample_Text.txt.enc
Enter encryption key (Hex): ede0633a204663db44319a1dc0d1139c
Enter encryption IV (Hex): 602c4bf244dda5c5bc9b2ada37de6593
Decrypting file...
File decrypted and saved as \Downloads\Programming\Python\AES Tool\Sample_Text.txt_decrypted.enc

```

FIG 6.5 FILE DECRYPTION

```

=====
          Chat Application
=====
Enter your encryption key (Hex): ede0633a204663db44319a1dc0d1139c
Start server or client? (s/c): s
Server started! Waiting for connections...
Accepted connection from ('127.0.0.1', 53155)
Type 'exit' to shut down the server: Received: Hello

```

FIG 6.6 CHAT APPLICATION – SERVER SIDE

```

=====
                        Chat Application
=====
Enter your encryption key (Hex): ede0633a204663db44319a1dc0d1139c
Start server or client? (s/c): c
Enter message to send (or 'exit' to quit): Hello
Encrypted Message (Hex): f5d4a3bd990cdd0aab92f4a48ccbd6764e74de3fe2f345902744
9e83fab4cce9
Encrypted Message (Base64): 9dSjvZkM3QqrkvSkjMvWdk503j/i80WQJ0Seg/q0z0k=
Enter message to send (or 'exit' to quit): █

```

FIG 6.7 CHAT APPLICATION – CLIENT SIDE

```

=====
                        AES Encryption/Decryption Tool
=====
1. Encrypt Text
2. Decrypt Text
3. Encrypt/Decrypt File
4. Chat Application
5. Help
6. Exit
Enter choice (1/2/3/4/5/6): 6

=====
                        Exiting...
=====

```

FIG 6.8 EXITING/TERMINATING

CHAPTER 7

CONCLUSION

The **SAFEBYTE** tool is a robust, user-friendly, and versatile program designed to secure sensitive data through encryption. It effectively uses **AES (Advanced Encryption Standard)** in **CBC mode**, ensuring confidentiality and data integrity. The program's features, including password-based key derivation, compression, and real-time encrypted communication, provide a comprehensive solution for text, file, and message encryption.

Key highlights of the tool:

1. **Security:** Strong encryption and secure key/IV generation protect against unauthorized access.
2. **Flexibility:** Supports both text and file encryption/decryption, catering to various needs.
3. **Usability:** A clear, interactive interface ensures accessibility for users with different skill levels.

With its easy-to-follow workflows and powerful features, this tool is a practical implementation of modern cryptographic principles for both personal and professional use cases. It ensures that sensitive data remains confidential, whether stored locally or shared across networks.

CHAPTER 8

APPENDIX (PROGRAM)

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Protocol.KDF import scrypt
from Crypto.Hash import SHA256
import base64
import binascii
import time
import os
import zlib
import socket
import threading
from colorama import Fore, init

# Initialize Colorama for colored output
init(autoreset=True)

def generate_key_iv():
    """Generate a new AES key (128 bits) and IV (128 bits)."""
    key = get_random_bytes(16)
    iv = get_random_bytes(16)
    return key, iv

def generate_key_from_password(password, salt):
    """Derive a key from a password using scrypt."""
```

```

return scrypt(password.encode(), salt, 16, N=2**14, r=8, p=1)

def pad_data(data):
    """Pad data to be a multiple of AES block size using PKCS#7."""
    padding_len = AES.block_size - len(data) % AES.block_size
    return data + bytes([padding_len] * padding_len)

def unpad_data(data):
    """Remove padding from data."""
    padding_len = data[-1]
    return data[:-padding_len]

def encrypt(plaintext, key, iv, mode):
    """Encrypt plaintext using AES with the provided key and IV."""
    cipher = AES.new(key, mode, iv)
    padded_data = pad_data(plaintext)
    ciphertext = cipher.encrypt(padded_data)
    return iv + ciphertext # Prepend IV to the ciphertext

def decrypt(ciphertext_with_iv, key, mode):
    """Decrypt ciphertext using AES with the provided key and IV."""
    iv = ciphertext_with_iv[:16] # Extract IV
    ciphertext = ciphertext_with_iv[16:] # Extract ciphertext
    cipher = AES.new(key, mode, iv)
    padded_data = cipher.decrypt(ciphertext)
    return unpad_data(padded_data)

def compress_data(data):
    """Compress data using zlib."""

```



```

return zlib.compress(data)

def decompress_data(data):
    """Decompress data using zlib."""
    return zlib.decompress(data)

def save_key_iv(key, iv, path):
    """Save key and IV to a file in hexadecimal format."""
    with open(path, 'wb') as f:
        f.write(binascii.hexlify(key) + b'\n' + binascii.hexlify(iv))

def load_key_iv(path):
    """Load key and IV from a file."""
    with open(path, 'rb') as f:
        key_hex, iv_hex = f.read().split(b'\n')
        key = binascii.unhexlify(key_hex)
        iv = binascii.unhexlify(iv_hex)
        return key, iv

def show_progress(message):
    """Display a simple progress indicator."""
    print(f' {Fore.YELLOW} {message} ', end='', flush=True)
    for _ in range(3):
        time.sleep(0.5)
        print(".", end='', flush=True)
    print() # Newline after progress

def print_header(title):
    """Print a formatted header for sections."""

```

```

print(f'\n{Fore.CYAN} {'=' * 50}')
print(f'{Fore.CYAN} {title.center(50)}')
print(f'{Fore.CYAN} {'=' * 50}')

```

```

def input_with_validation(prompt, validator=None):
    """Prompt user for input with optional validation."""
    while True:
        value = input(prompt).strip()
        if validator:
            try:
                validator(value)
                return value
            except ValueError as e:
                print(f'{Fore.RED}Error: {e}')
        else:
            return value

```

```

def validate_hex(value):
    """Check if the given string is a valid hexadecimal."""
    if not all(c in "0123456789abcdefABCDEF" for c in value):
        raise ValueError("Invalid hexadecimal characters.")

```

```

def validate_key_iv_length(value):
    """Ensure the key or IV is 16 bytes (128 bits) long."""
    if len(binascii.unhexlify(value)) != 16:
        raise ValueError("Key and IV must be 16 bytes long.")

```

Encryption and decryption functions for files

```

def encrypt_file(file_path, key, iv, mode, save_key_iv_path=None):

```

```

"""Encrypt a file with optional compression."""
try:
    with open(file_path, 'rb') as f:
        data = f.read()
        compressed_data = compress_data(data)
        show_progress("Encrypting file")
        encrypted_data = encrypt(compressed_data, key, iv, mode)
        encrypted_file_path = file_path + ".enc"
        with open(encrypted_file_path, 'wb') as f:
            f.write(encrypted_data)
        print(f" {Fore.GREEN}File encrypted and saved as
{encrypted_file_path}")
        if save_key_iv_path:
            save_key_iv(key, iv, save_key_iv_path)
except Exception as e:
    print(f" {Fore.RED}Error during file encryption: {e}")

def decrypt_file(file_path, key, mode):
    """Decrypt a file with optional decompression."""
    try:
        encrypted_file_path = file_path if file_path.endswith('.enc') else file_path +
'.enc'
        with open(encrypted_file_path, 'rb') as f:
            ciphertext_with_iv = f.read()
            show_progress("Decrypting file")
            decrypted_data = decrypt(ciphertext_with_iv, key, mode)
            decompressed_data = decompress_data(decrypted_data)
            decrypted_file_path = file_path.rsplit('.', 1)[0] + "_decrypted" +
os.path.splitext(file_path)[1]
            with open(decrypted_file_path, 'wb') as f:

```

```

        f.write(decompressed_data)

    print(f'{Fore.GREEN}File decrypted and saved as
{decrypted_file_path}')

except Exception as e:

    print(f'{Fore.RED}Error during file decryption: {e}')

# Encrypt and decrypt text functions
def encrypt_text():
    """Encrypt user-provided plaintext."""
    print_header("Encryption")
    plaintext = input_with_validation("Enter plaintext to encrypt: ").encode()
    key, iv = generate_key_iv()
    mode = AES.MODE_CBC
    show_progress("Encrypting")
    ciphertext_with_iv = encrypt(plaintext, key, iv, mode)
    print(f'{Fore.GREEN}\nEncryption Details:')
    print(f'Encryption Key (Hex):
{Fore.CYAN} {binascii.hexlify(key).decode()}')
    print(f'Encryption IV (Hex): {Fore.CYAN} {binascii.hexlify(iv).decode()}')
    print(f'Ciphertext (Hex):
{Fore.CYAN} {binascii.hexlify(ciphertext_with_iv).decode()}')
    print(f'Ciphertext (Base64):
{Fore.CYAN} {base64.b64encode(ciphertext_with_iv).decode()}')

def decrypt_text():
    """Decrypt user-provided ciphertext."""
    print_header("Decryption")
    ciphertext_base64 = input_with_validation("Enter ciphertext (Base64) to
decrypt: ")
    key_hex = input_with_validation("Enter encryption key (Hex): ",
validate_hex)

```

```

iv_hex = input_with_validation("Enter encryption IV (Hex): ", validate_hex)

try:
    validate_key_iv_length(key_hex)
    validate_key_iv_length(iv_hex)

    ciphertext_with_iv = base64.b64decode(ciphertext_base64)
    key = binascii.unhexlify(key_hex)
    iv = binascii.unhexlify(iv_hex)
    mode = AES.MODE_CBC

    show_progress("Decrypting")
    decrypted_text = decrypt(ciphertext_with_iv, key, mode)
    print(f'{Fore.GREEN}\nDecryption Result:')
    print(f'Decrypted Plaintext: {Fore.CYAN} {decrypted_text.decode()}')

except (binascii.Error, ValueError) as e:
    print(f'{Fore.RED}Error: {e}')
    print(f'{Fore.RED}Ensure that the key and IV are valid hexadecimal strings and the ciphertext is correctly encoded in Base64.')
except Exception as e:
    print(f'{Fore.RED}Unexpected error: {e}')

def show_help():
    """Display help information."""
    print_header("Help")
    print(f'{Fore.GREEN}AES Encryption/Decryption Tool Usage:')
    print(f'{Fore.GREEN}1. Encrypt Text:')
    print(f' - Enter the plaintext to be encrypted.')

```

```

    print(f" - The tool will generate a new AES key and IV, encrypt the text, and
display the encrypted text in both Hex and Base64 formats.")

    print(f"{Fore.GREEN}2. Decrypt Text:")

    print(f" - Enter the Base64-encoded ciphertext, the AES key, and the IV
used for encryption.")

    print(f" - The tool will decrypt the ciphertext and display the original
plaintext.")

    print(f"{Fore.GREEN}3. Encrypt/Decrypt File:")

    print(f" - Select encryption or decryption.")

    print(f" - Enter the file path and optionally save the key and IV.")

    print(f"{Fore.GREEN}4. Chat Application:")

    print(f" - Start a chat server or client to send encrypted messages.")

    print(f"{Fore.GREEN}5. Help:")

    print(f" - Display this help message.")

    print(f"{Fore.GREEN}6. Exit:")

    print(f" - Exit the tool.")

```

Chat functionality

```

def handle_client(client_socket, key):
    """Handle incoming messages from the client."""
    while True:
        try:
            message = client_socket.recv(1024)
            if not message:
                break
            iv = message[:16] # Extract IV
            ciphertext = message[16:] # Extract ciphertext
            decrypted_message = decrypt(ciphertext, key, AES.MODE_CBC)
            print(f"{Fore.GREEN}Received:
{Fore.CYAN} {decrypted_message.decode()}")

```

```

except Exception as e:
    print(f'{Fore.RED}Error handling message: {e}')
    break
client_socket.close()

def start_server(key):
    """Start the chat server."""
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(("0.0.0.0", 9999)) # Bind to all interfaces on port 9999
    server.listen(5)
    print(f'{Fore.GREEN}Server started! Waiting for connections...')

    while True:
        client_socket, addr = server.accept()
        print(f'{Fore.GREEN}Accepted connection from {addr}')
        client_handler = threading.Thread(target=handle_client,
args=(client_socket, key))
        client_handler.start()

    # Exit option for server
    exit_command = input("Type 'exit' to shut down the server: ")
    if exit_command.lower() == 'exit':
        print(f'{Fore.YELLOW}Shutting down server...')
        server.close()
        break

def chat_client(key):
    """Start the chat client."""
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

```

```

client.connect(("127.0.0.1", 9999)) # Connect to the server

while True:
    message = input("Enter message to send (or 'exit' to quit): ")
    if message.lower() == "exit":
        client.close()
        break

    iv = get_random_bytes(16) # Generate a new IV for each message
    encrypted_message = encrypt(message.encode(), key, iv,
AES.MODE_CBC)

    # Print the encrypted message in Hex and Base64 before sending
    print(f'{Fore.GREEN}Encrypted Message (Hex):
{Fore.CYAN}{binascii.hexlify(encrypted_message).decode()}')

    print(f'{Fore.GREEN}Encrypted Message (Base64):
{Fore.CYAN}{base64.b64encode(encrypted_message).decode()}')

    client.send(iv + encrypted_message) # Send IV and encrypted message

def chat_interface():
    """Provide an interface for chat functionality."""
    print_header("Chat Application")
    key_hex = input_with_validation("Enter your encryption key (Hex): ",
validate_hex)
    key = binascii.unhexlify(key_hex)

    operation = input_with_validation("Start server or client? (s/c): ", lambda x: x
in {'s', 'c'})
    if operation == 's':
        start_server(key)

```



```

elif operation == 'c':
    chat_client(key)

# Main function to run the tool
def main():
    while True:
        print_header("AES Encryption/Decryption Tool")
        print(f"{Fore.GREEN}1. Encrypt Text")
        print(f"{Fore.GREEN}2. Decrypt Text")
        print(f"{Fore.GREEN}3. Encrypt/Decrypt File")
        print(f"{Fore.GREEN}4. Chat Application")
        print(f"{Fore.GREEN}5. Help")
        print(f"{Fore.GREEN}6. Exit")

        choice = input_with_validation("Enter choice (1/2/3/4/5/6): ", lambda x: x
in {'1', '2', '3', '4', '5', '6'})

        if choice == '1':
            encrypt_text()
        elif choice == '2':
            decrypt_text()
        elif choice == '3':
            print_header("File Encryption/Decryption")
            operation = input_with_validation("Encrypt or decrypt file? (e/d): ",
lambda x: x in {'e', 'd'})
            file_path = input_with_validation("Enter file path: ")
            if not os.path.isfile(file_path):
                print(f"{Fore.RED}Error: File does not exist.")
                continue

```

```

if operation == 'e':
    key, iv = generate_key_iv()
    save_key_iv_path = input_with_validation("Enter path to save key
and IV (or press Enter to skip): ") or None
    encrypt_file(file_path, key, iv, AES.MODE_CBC, save_key_iv_path)
elif operation == 'd':
    key_hex = input_with_validation("Enter encryption key (Hex): ",
validate_hex)
    iv_hex = input_with_validation("Enter encryption IV (Hex): ",
validate_hex)
    try:
        validate_key_iv_length(key_hex)
        validate_key_iv_length(iv_hex)
        key = binascii.unhexlify(key_hex)
        iv = binascii.unhexlify(iv_hex)
        decrypt_file(file_path, key, AES.MODE_CBC)
    except (binascii.Error, ValueError) as e:
        print(f'{Fore.RED}Error: {e}')
        print(f'{Fore.RED}Ensure that the key and IV are valid
hexadecimal strings.")
    except Exception as e:
        print(f'{Fore.RED}Unexpected error: {e}')

elif choice == '4':
    chat_interface() # Call chat interface

elif choice == '5':
    show_help()

elif choice == '6':

```

```
print_header("Exiting...")  
break
```

```
else:
```

```
    print(f' {Fore.RED}Invalid choice, please select 1, 2, 3, 4, 5, or 6.")
```

```
if __name__ == "__main__":
```

```
    main()
```

REFERENCES

1. **"Security Enhancement of AES-CBC and its Performance Evaluation Using the Avalanche Effect"** This paper explores improving AES-CBC encryption and measures its robustness through the avalanche effect. It provides insights into enhancing security in practical applications.
2. **"Analysis of Encryption Techniques for Secure Communication"** A comparative study of encryption techniques, including AES, focusing on secure communication in various environments. It highlights the strengths of AES for practical use.
3. **"A Comparative Analysis of AES Common Modes of Operation"** This research examines different AES modes, including CBC, detailing their efficiency and use cases for secure data transmission.
4. **"Some AES CBC Encryption Myth-busting"** Discusses misconceptions about AES-CBC and provides insights into its application and potential for optimization in secure systems.
5. **"AES in Cipher Block Chaining Mode for Resource-Constrained Devices"** Focused on applying AES-CBC in resource-constrained environments, such as IoT, ensuring robust encryption without heavy computational overheads.
6. **"Data Compression and Encryption Techniques for Secure Communication"** Highlights methods to combine data compression and encryption to optimize storage and communication, aligning with SafeByte's features.
7. **"Real-Time Secure Communication with AES-CBC"** This study explores real-time secure communication applications using AES-CBC, relevant to SafeByte's chat functionality.
8. **"Advanced Cryptographic Techniques for End-to-End Secure Communication"** Discusses secure communication systems integrating AES and other modern cryptographic approaches to ensure data integrity and confidentiality.